

RESEARCH ARTICLE

CricShot10k: A Large-Scale Video Dataset for Cricket Shot Classification

MUBTASIM KAMAL DIHAN^{ID}, ABDULLAH^{ID}, AMINA^{ID}, AND SABBIR AHMED^{ID}

Department of Computer Science and Engineering, Islamic University of Technology, Gazipur 1704, Bangladesh

Corresponding author: Sabbir Ahmed (sabbirahmed@iut-dhaka.edu)

ABSTRACT Cricket, one of the most popular sports worldwide, involves a wide variety of batting shots that directly affect match outcomes and play a central role in analyzing player performance. In this context, our study focuses on building an extensive dataset for cricket shot classification from videos and addressing the challenges of existing approaches. We observe that previous works suffer from the time-consuming task of manually trimming match videos into individual shot clips, which has led to relatively small datasets. To overcome this limitation, we propose a novel approach that leverages recurring patterns in cricket broadcasts to automatically split long videos into multiple shot clips. This reframes the task into a form similar to text or image annotation, thereby facilitating the construction of large-scale datasets. Using this method, we build a dataset, called CricShot10k, consisting of 10,086 video clips across 15 shot classes. Our dataset is nearly eight times larger and more diverse than existing ones, covering shots from all formats, and being the first to include women's and under-19 matches, as well as older matches dating back to 1996. Building on this dataset, we present a modified cricket shot classification framework, CricShotNet, which incorporates two new layers: cropping and segmentation. To identify the best-performing architecture for the subsequent layers, we conduct an extensive comparison across 15 deep feature extractors and 9 variations of temporal feature extractors using GRU, LSTM, and BiLSTM. Our experiments show that EfficientNetV2-S combined with a GRU of 128 units achieve the highest accuracy of 89% on CricShot10k and outperform other state-of-the-art approaches on existing cricket shot video datasets, thus demonstrating the effectiveness of the proposed framework. Our codes and dataset are available at <https://github.com/Dihan69/CricShot10k>

INDEX TERMS Action recognition, cricket shot classification, cricket shot video dataset, EfficientNetV2, GRU, sports analytics, video classification.

I. INTRODUCTION

Cricket is widely regarded as the second most popular sport, just behind football, according to the number of fans [1]. Although a good number of cricket fans are concentrated in highly populated South Asian countries, the recent conclusion of the T20 World Cup 2024 in the United States shows the ongoing efforts to popularize cricket all over the world. With the increasing number of cricket matches worldwide and the introduction of new franchise leagues, it has become increasingly challenging for cricket commentary and analytics websites like CricBuzz and ESPNcricinfo to rely solely on manual entries. Even now,

a high number of cricket matches are provided only with score updates even if the game is broadcast live. To overcome this limitation, an appropriate automated commentary generation model is required that will be capable of detecting different types of event with high accuracy from live broadcast camera angles, including shot type, ball's line and length, and the ball's trajectory after the impact with bat.

Among these, accurately determining the type of shots requires developing a reliable shot classification model with broad applicability, which is the primary focus of this work. In cricket, the shots played by a batter are central to the game and influence almost every event on the field. Having an automated shot classifier that functions accurately in all types of cricket matches will provide players with a rich and diverse repertoire of shots. Using this, batters can identify

The associate editor coordinating the review of this manuscript and approving it for publication was Prakasam Periasamy^{ID}.

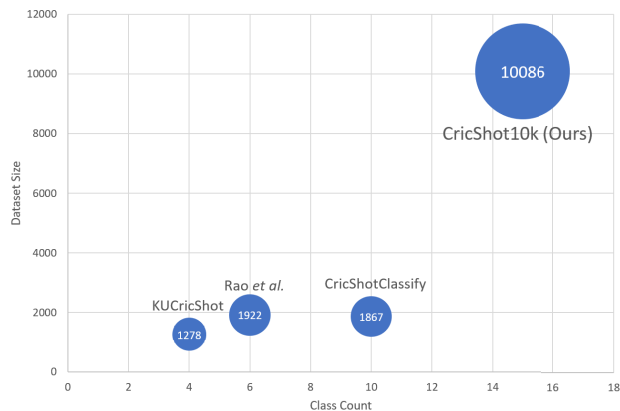


FIGURE 1. Illustration of the superiority of our dataset, CricShot10k. Compared to previous works on cricket videos, our dataset contains more classes and shot types. Additionally, it is the most diverse dataset, covering all eras, formats, match types, and even including women's and under-19 matches.

the most effective way to play each shot and pinpoint their mistakes. Bowlers can identify error-prone areas to target the weaknesses of a batter. In addition, cricket highlights are currently created manually, based completely on the time period in which shots are played. An automated shot classifier could significantly streamline this process. This significant importance of shots in cricket has led us to prioritize them as the central aspect of our study.

Accurate classification of cricket shots requires analyzing video sequences rather than single frames. Frame-based approaches [2], [3] using Convolutional Neural Network (CNN) limit most potential use cases and are unreliable for accurate prediction. Hence, the availability of large and diverse video datasets is crucial to develop models capable of reliable shot recognition. The few existing studies on cricket videos and their datasets, as shown in Figure 1, suffer from various limitations. Hoque et al. [4] developed KUCricshot, a dataset of 1,278 videos covering four types of shots. However, it is restricted to the test format and only includes shots played by right-handed batters. Similarly, Sen et al. [5] introduced CricShotClassify, a dataset of 1,867 videos spanning 10 classes. Many of these videos are unnecessarily long, and several were artificially generated by horizontally flipping clips to represent opposite-handed batters.

The manual process of trimming individual shot videos from full match videos is a time-intensive task, which is why most existing cricket datasets are relatively small and often contain noise. To overcome this challenge, we devised an effective yet simple approach using multiple fine-tuned YOLOv11 [6] object detection models and logical rules based on recurring patterns during the execution of a cricket shot to automatically trim each match highlights video into several one-second long shot videos. By automating this trimming process, the task was simplified into an annotation workflow similar to that used for text or images.

After annotation, we classified 10,086 shot videos into 15 distinct types: Cover Drive, Defensive, Down the Wicket,

Flick, Hook, Late Cut, Lofted Legside, Lofted Offside, Pull, Reverse Sweep, Scoop, Square Cut, Straight Drive, Sweep and Upper Cut. Compared to previous works, our dataset, CricShot10k, is nearly eight times larger, includes more shot types, and offers greater representation within each class. It was compiled from 536 highlights videos, covering international and domestic men's and women's cricket, as well as Under-19 tournaments. Notably, our work is the first to include matches beyond men's senior cricket, ensuring a more diverse and inclusive representation of the sport. We also included matches from all formats and tournament types, as well as older matches dating back to 1996.

The classification of cricket shots depends mainly on the batter's action, but live broadcast videos include surrounding areas that do not affect the prediction of shot type. This highlights the need for a dedicated model to reduce noise and improve generalization, an aspect largely overlooked in prior work. Our model, CricShotNet, introduces two additional layers, cropping and segmentation, before the standard deep and temporal feature extractors. We first crop the region around the striker and bat using fine-tuned YOLOv11 object detection models to eliminate irrelevant elements in each frame. The cropped frames then are resized to 224×224 , and 15 uniformly spaced frames are selected. To reduce the effect of jersey color and lighting variations, a semi-transparent overlay is applied to the batter and bat. Frames are processed by EfficientNetV2-S [7], identified as the best among 15 CNN feature extractors. The extracted features are flattened and passed to a GRU layer with 128 hidden units, chosen after testing nine variants including GRU, LSTM, and BiLSTM. Finally, the GRU output is fed into a classifier followed by a softmax layer to produce the prediction. CricShotNet achieved a top-1 accuracy of 89%, along with top-2 and top-3 accuracies of 97% and 99%, respectively, on the CricShot10k dataset. It also outperformed state-of-the-art approaches on CricShot10k as well as other existing datasets, confirming its effectiveness.

In summary, our contributions are threefold.

- 1) We created an efficient automated data collection method for cricket shot videos to facilitate the creation of large-scale datasets and simplify the annotation process.
- 2) We annotated and built a dataset, CricShot10k, of 10,086 cricket shot videos across 15 classes from diverse match types, establishing it as the largest and most inclusive cricket shot dataset to date.
- 3) We added a custom cropping and segmentation layer atop baseline video classification layers to enhance cricket shot classification. Extensive experiments showed EfficientNetV2-GRU as the best-performing baseline architectures for the proposed CricShotNet framework.

The rest of the paper is organized as follows. In Section II, we provide a literature review and identify the research gaps

in related works to justify our motivation. Section III presents the construction methodology and analysis of our dataset. The details of the proposed methodology are presented in Section IV. In Section V, we provide qualitative and quantitative analyses to justify the efficacy of our proposed method. Finally, Section VI offers concluding remarks, discusses our limitations, and suggests directions for future research.

II. RELATED WORKS

This section provides a brief overview of related work, organized into four categories. First, we review machine learning approaches in other sports, followed by general cricket-related studies beyond shot classification. We then discuss image-based cricket shot classification works and finally review existing cricket video datasets.

A. GENERAL WORKS ON OTHER SPORTS

Before the emergence of machine learning and Artificial Intelligent based models, substantial sports analytics-based research had been conducted, particularly in areas such as player performance analysis [8], [9], [10]. The first machine learning applications in sports focused on predicting the results and outcomes of games [11], [12], [13], [14]. The introduction of deep learning has facilitated more accurate and comprehensive analysis in sports. Real-time object tracking techniques have been applied in basketball to analyze the relationships between passes made during the game [15]. The target tracking method has been combined with a deep learning-based model to track football players [16]. Human pose estimation techniques have been used in baseball to determine whether a batsman executed a good bat swing or not [17]. Ganser et al. [18] has built a dataset and has used a deep neural network to classify various types of tennis shots.

B. GENERAL WORKS ON CRICKET

Beyond shot classification, several studies have explored cricket video analysis for various purposes. An early 2011 study proposed a hierarchical key-frame approach for semantic event detection and classification [19]. Around the same time, another study proposed a hierarchical key-frame approach that distinguishes real-time and replay frames, field views, close-ups, and crowd scenes [20]. Besides, Bird Swarm Optimization based autoencoder techniques have been applied to detect and classify umpires on the cricket field [21]. Shot boundary detection and replay segment extraction have been employed to summarize cricket videos efficiently [22]. Computer vision methods have also been combined with large language models to generate cricket highlights using a multimodal approach [23]. A recent survey by Raval et al. [24] provided a comprehensive overview of state-of-the-art strategies for event-detection-based cricket video summarization.

C. IMAGE-BASED CRICKET SHOT CLASSIFICATION

A number of cricket based works have used images of batters playing shots and 2D CNN models for shot classification [2], [3]. Another work combined Support Vector Machine (SVM) with CNN for classification [25], while Dey et al. [26] used a more recent vision transformer model. However, such single image approaches restrict recognition to a small set of cricket shots. Limiting classification to static images instead of full video sequences reduces the scope of applications, as they fail to capture bat movement, which is crucial for accurate execution and technical analysis. While certain shots may be identifiable from single frames, selecting the decisive frame requires manual intervention, making fully automated classification impractical. Furthermore, motion vector based methods [27] are unsuitable for live broadcast videos, where the camera moves dynamically with play.

D. VIDEO-BASED CRICKET SHOT CLASSIFICATION

Research on video-based cricket shot classification remains limited, with only a few works attempting to build dedicated cricket video datasets. KUCricShot [4] introduced a dataset of 1,278 videos covering 4 types of shots, each about one second long, recorded from live broadcast perspectives. However, their proposed model used the spatial temporal capabilities of a Long-Term Recurrent Convolutional Network (LRCN) [28], achieving only 73% accuracy. Besides, the dataset is restricted to the test format and right handed batters, and in our replication, the trained model failed to generalize to other formats, particularly limited-over cricket where jersey colors differ.

Sen et al. [5], in their work CricShotClassify, created a dataset of 1,867 videos across 10 shot classes. This dataset addresses some KUCricShot limitations but introduces new issues: clip durations of up to 7 seconds are unnecessarily long, and several shots are mislabeled based on ball trajectory rather than the intended shot. Moreover, a substantial portion of the videos are horizontally flipped to create samples for opposite-handed batters. This common data augmentation should not be included in total dataset size, effectively halving the actual dataset. More concerning is potential overlap of samples in training and test sets, which could artificially inflate performance and may have contributed to the reported 93% accuracy of the VGG16-GRU model.

In a recent work [29], a body pose landmark approach was applied to a dataset of 1922 clips from another study [30], covering six shot classes. After a batsman detection step, they used Mediapipe pose detection to extract landmarks. Pose-based representations eliminate important visual cues, such as texture, appearance, and context, compressing full-color frames into sparse skeletal structures. This loss of fine-grained information can limit classification accuracy, particularly for visually similar shots. They reported 80% accuracy with all classes, rising to 93% after reducing classes and videos. However, the extent of this reduction was not clearly specified. In general, fewer classes and a

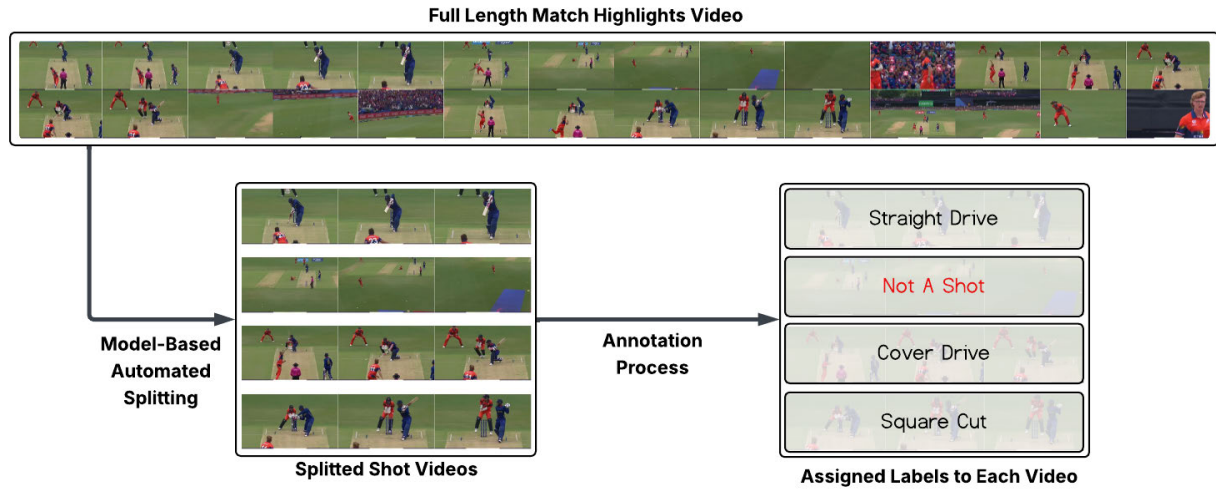


FIGURE 2. Illustration of our process of building the dataset. We first collected full length highlight videos, each containing multiple shots. Using our proposed automated method, we split the videos into multiple shot clips. The split videos are then annotated with appropriate class labels. Shots that were incorrectly split are labeled as not a shot and are excluded from the dataset.

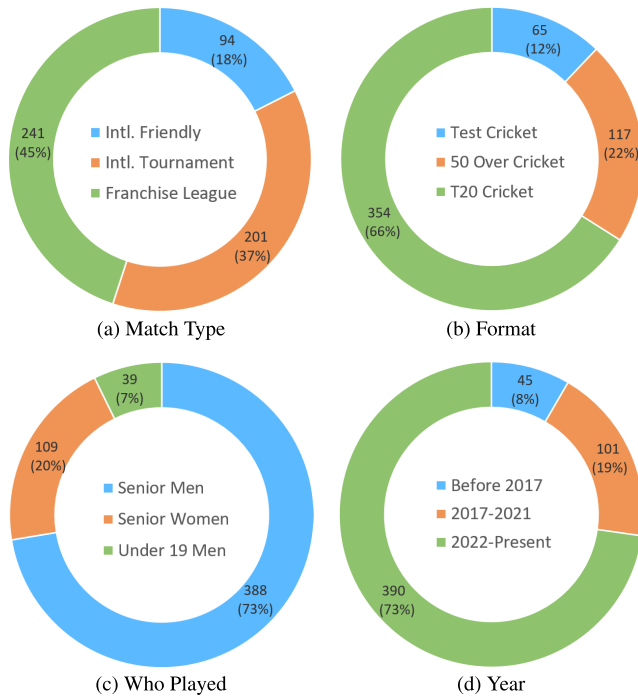


FIGURE 3. Characteristics of all 536 collected match videos based on 4 different categories. (Intl. stands for international).

smaller dataset make the task less complex. The authors noted potential errors in the dataset, which we could not verify due to the lack of access of the dataset. In contrast, we show that our segmentation based method achieves superior performance, especially as the dataset size increases.

III. DATASET: CRICSHOT10K

In this section, we first outline the complete methodology for constructing our shot dataset, CricShot10k. The method involves three key steps: (a) collecting full-length match

videos, (b) segmenting each match video into individual shot clips, and (c) labeling each shot clip. Figure 2 illustrates this process of building the dataset. Final parts of this section highlight the characteristics of our dataset and discuss the data augmentation process.

A. MATCH VIDEO COLLECTION

In this study, a total of 536 highlight videos were collected, with durations ranging from a minimum of 5 minutes to a maximum of 36 minutes. These highlights videos speed up our automated shot-splitting by excluding segments unrelated to shots. Previous research on cricket videos has focused exclusively on men's senior cricket. However, for practical applications, the model should be capable of handling all forms of televised cricket. To ensure broader applicability, we have included videos from women's and Under-19 cricket as well. Additionally, to demonstrate that our shot data splitting method is effective across different eras, we have incorporated older videos, including matches dating back to the 1996 Cricket World Cup. Matches before this period are rarely available online in usable quality. The characteristics of these videos are summarized in Figure 3. The distribution reflects the actual frequency of matches played worldwide. For example, T20 matches are more common than 50 over One Day International (ODI) and Test matches. The complete list of matches is available in the GitHub repository, and each video is sequentially numbered from 1 to 536 for easier identification.

B. SHOTS DATA SPLITTING

Each match highlight video contains multiple deliveries during which different types of shots are played. In addition to the shots, these videos also include various other segments, such as the ball's trajectory, player celebrations and reactions and more. From each highlight video, a large number of

individual shot videos can be extracted and varying labels can be assigned to them. With advances in computer vision and video classification, the recognition of actions in short segments of long videos is becoming increasingly efficient and reliable [31]. However, recognizing sports events such as cricket shots requires large, specialized datasets whose creation demands substantial dedicated research. Therefore, we have instead chosen a custom object detection and logic-based approach, taking advantage of the general patterns observed when a batter plays a shot. This approach requires minimal effort to develop, yet performs outstandingly.

In cricket terminology, the batter currently facing the ball delivery is referred to as the striker [32]. This distinction is needed to identify instances in which the batter is not facing the delivery. In general, the striker plays a shot immediately after the bowler releases the ball. Before release, the ball remains hidden in the bowler's hand and becomes visible only when the bowler lets it go. This makes the appearance of the ball a potential start point for trimming. However, after the shot, cameras follow the ball, which may briefly disappear and reappear due to manual operation of the camera and small size of the ball. This could cause false shot detections and increase annotation effort. Hence, ball detection alone is insufficient. When the ball is released, both the striker and the ball are visible, but once the camera follows the ball, the striker usually disappears. Thus, defining the start as the moment both are visible helps avoid mistaking random ball appearances for new shots. Although this method proved to be highly effective, adding additional constraints of bowler's presence allowed us to avoid rare errors. Figure 4 shows an example of a video that could have been incorrectly detected as a shot if not for the additional bowler constraint.

To implement this, we created a dataset of 602 images and annotated them to detect various types of individuals on the cricket field in 7 classes: striker, non-striker, batter, bowler, fielder, umpire, keeper, and others. The distinction between these individuals is possible due to specific postures or camera angles. The batter class is used to identify individuals holding a bat but not in the posture of a striker or non-striker. Although we primarily need to detect strikers and bowlers, we included other classes with distinct features to avoid misclassification of unseen individuals as strikers or bowlers. A separate dataset consisting of 1,400 images was created for ball detection. Due to the ball being a small object, we used a larger and more challenging dataset for this purpose. To annotate the bounding boxes for these object detection datasets, the CVAT online platform¹ was used. Both datasets were used separately to fine-tune the pre-trained YOLO11m model [6], with 20% set aside for testing.

To ensure robustness and avoid the possibility of coincidental detections, we only considered a frame containing striker, bowler and ball as the starting point if at least 5 out of the next 10 frames also contained the striker or the ball. For these subsequent frames, the bowler was not considered,

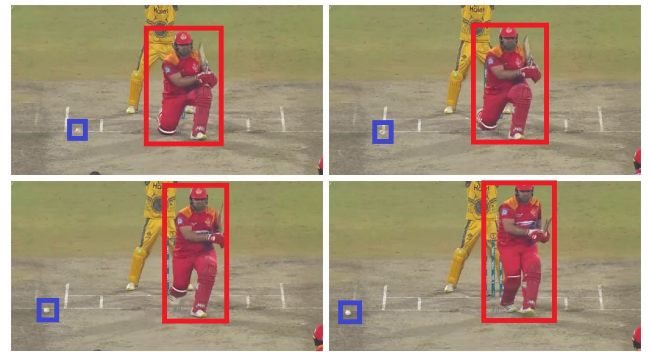


FIGURE 4. Illustration of the benefit of incorporating bowler detection constraint in our automated shot splitting method. In this video, the striker (red bounding box) and the ball (blue bounding box) are visible, but the bowler is absent. This indicates the clip is not a valid shot. Without the bowler constraint, it would be incorrectly trimmed as a shot.

as the bowler may not be visible when the camera zooms in on the striker. Starting from the first frame, we took exactly 1 second before ending the video, which we have chosen to be the perfect duration after multiple trials. After this 1 second, the search for the next shot in the video restarts following the same process. Appendix A provides a clearer illustration of this methodology.

C. SHOTS DATA ANNOTATION

To ensure precise labeling, specialized tasks like cricket shot classification require annotators with sufficient domain expertise. To address this, we employed a small group of three annotators with extensive cricket knowledge using an approach inspired by Chang et al. [33]. The three annotators were tasked with sorting all videos into one of 15 shot classes, along with three additional labels: uncertain, uncategorized, and not a shot. The *uncategorized* label was for videos that did not fit into any of the 15 predefined shot classes. The uncertain category was reserved for videos that the annotators believed might belong to one of the 15 classes but were uncertain about. Finally, the *not a shot* category was included to account for videos mistakenly identified as shots during automated shot splitting. Such errors can arise from inherent limitations of object detection models, which may misinterpret complex backgrounds [34]. Although modern techniques are generally effective, variations in lighting and camera angles can degrade detection accuracy [35]. As a result, perfect performance cannot always be guaranteed in scenarios involving digital capture [36].

After independent labeling, videos with at least two matching labels were assigned to the corresponding class. Videos with completely conflicting labels or marked uncertain were reviewed through annotator discussion. Following this discussion, if two of the three annotators reached consensus, the agreed label was assigned to the video. The remaining unresolved videos were placed in the *uncategorized* category. Although *not a shot* and *uncategorized* clips were excluded from our dataset, we have provided them in separate folders in our repository so they can be used in future studies

¹<https://www.cvat.ai>



FIGURE 5. Three selected frames from one video of each of the 15 classes, along with an *uncategorized* example. This uncategorized shot was chosen to illustrate the potential value of adding more class labels. Since the batter deliberately avoids playing a shot in this video, it could be labeled as a *leave* shot. We excluded such shots from the dataset due to their rarity in collected highlight videos.

to detect all cricket events or to add more class labels. Table 1 shows the number of videos in different phases of the annotations. The automated method originally produced 14,369 clips, from which we excluded more than 4,300 for various reasons. By going through this rigorous annotation process, despite resulting in the reduction of a good chunk of samples, we ensured that every clip included in CricShot10k meets the highest standards of quality in all aspects.

To quantitatively understand the impact of annotation quality on dataset reliability, we analyze Fleiss' Kappa, a widely-used metric for assessing inter-annotator agreement [37]. For three annotators, the Fleiss' Kappa score is 0.9376, which falls within the perfect agreement range

(0.80–1.00) [38]. This indicates highly consistent annotations, which is crucial for constructing a stable dataset with low sensitivity to noise and subjectivity. The high agreement is partly due to selecting only videos where at least two of the three annotators agreed. Even among these 10,086 videos, all three annotators assigned the same label to 9,218 videos, while only 868 exhibited disagreement. This consistency arises because cricket shot types are well-defined and generally easy to identify for annotators with domain knowledge. Minor disagreements mainly occur due to gameplay dynamics, where imperfect execution, poor shot selection, or unexpected ball behavior can lead to borderline cases. Overall, the strong inter-annotator

TABLE 1. Summary of the number of videos in different phases of the data annotation process.

Description	Count
Total shot videos after automated splitting	14,369
Videos labeled same by at least two annotators	9,420
Videos required to be discussed	1,420
Videos labeled after discussion	666
Videos remained uncategorized after discussion	754
Total uncategorized shots after all process	1,840
Videos labeled as not a shot	2,443
Total shots used in dataset	10,086

agreement supports the high quality and practical usability of the dataset.

D. SHOT DATASET ANALYSIS

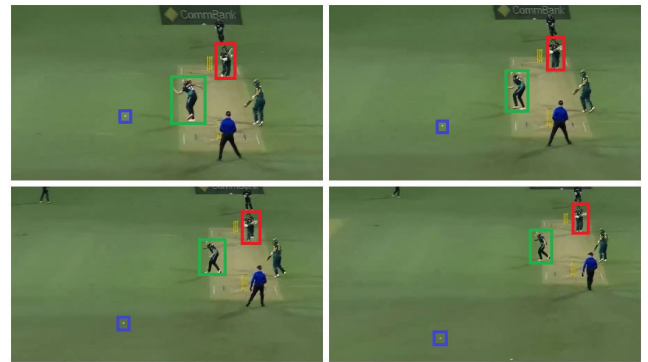
The annotation process resulted in a total of 10,086 video clips belonging to 15 classes. As each highlight video was unique, every clip contains shots from different balls with no duplicates. Three selected frames of a video from each class are shown in Figure 5. As of today, we know this to be the largest dataset with maximum number of classes. Figure 6 provides an example of a video that is *not a shot* but identified by our data collection method as a potential shot. Additional cases where the method misidentified a video as a shot involve slow-motion replays and Decision Review System (DRS). In rare instances, we excluded a video even if it contained an actual shot, when the model captured the first frame too early or too late, leaving insufficient context to determine shot type. All these *not a shot* videos account for only 17% of all collected clips (Table 1). Hence, we did not introduce additional automation to eliminate these anomalies, as the time consumed by the annotators to filter these videos is minimal, compared to the time required to design additional methods. Examples of these discarded clips are provided in Appendix B.

The data distribution of the shots in each class is shown in Table 2. It reflects both how shots are played in cricket and how highlight videos are prepared, with run-scoring shots like lofted, pull, cover drive and square cut shots being more common. We have two different types of shots: lofted offside and lofted legside, under the general category of lofted shots based on which side of the ground the ball is intended to be sent. This distinction was made because bat swing is quite different for these two types and categorizing them separately helps avoid imbalancing the overall distribution of other classes. If an application does not require the side-based separation, it can treat both types as a single class. However, this separation opens up potential for many unique use cases. For example, with this, it is possible to analyze when it is better to loft a delivery to a particular side of the ground.

On the other hand, shots such as the reverse sweep, upper cut, and scoop are played less frequently due to their higher risk and are generally considered unorthodox in cricket. This

TABLE 2. Distribution of samples in our dataset: CricShot10k.

No	Class label	Count	Percentage
1	Cover Drive	793	7.85%
2	Defensive	587	5.81%
3	Down The Wicket	844	8.36%
4	Flick	947	9.38%
5	Hook	532	5.27%
6	Late Cut	325	3.22%
7	Lofted Legside	1094	10.83%
8	Lofted Offside	1054	10.44%
9	Pull	769	7.62%
10	Reverse Sweep	257	2.55%
11	Scoop	279	2.76%
12	Square Cut	1000	9.89%
13	Straight Drive	419	4.15%
14	Sweep	905	8.97%
15	Upper Cut	281	2.78%
	Total shots	10,086	100%

**FIGURE 6.** Example of *not a shot* that is incorrectly trimmed as a shot video. In this video, the striker (red bounding box), the bowler (green bounding box), and the ball (blue bounding box) are all visible throughout, but no actual shot is played.

is reflected in the smaller number of instances we were able to collect for these shots from an equivalent set of videos. Therefore, the relative overrepresentation of attacking and run-scoring shots is not a consequence of using highlight videos, but rather reflects the natural dynamics of cricket. The only shot type that may be slightly underrepresented due to the use of highlight videos is defensive shots. However, these shots constitute 5.81% (587 out of 10,086) of the total, which is near the uniform distribution percentage.

It is evident that a slight class imbalance exists in the dataset. Due to imbalance, model accuracy may appear high even if performance on smaller classes is poor [39]. However, its effect is minimal when the class imbalance ratio, the ratio of the largest to smallest class, remains below a certain threshold [40]. Our model is on the lower side of the class imbalance ratio (4.26), and so the effect of imbalance can be considered minimal. Several research domains routinely deal with imbalanced datasets due to the inherent characteristics of the data [41], [42]. For example, in leaf disease classification from images, some diseases occur rarely, resulting in a



(a) Cover Drive (Top) and Straight Drive (Down)



(b) Pull (Top) and Hook (Down)

FIGURE 7. Illustration of high inter-class similarity in our dataset.

(a) Upper Cut



(b) Scoop

FIGURE 8. Illustration of high intra-class variation in our dataset.

reduced number of available samples for these diseases [43], [44], [45]. Similarly, in cricket, the imbalanced distribution of shots reflects the natural flow of the game rather than a sampling bias. With this limitation in mind, models designed for imbalanced datasets must be constructed carefully to handle class imbalance effectively. Different augmentation and classification techniques can also be used to address this problem [46], [47], [48]. We show in Section V that the class imbalance does not significantly affect model performance due to the low imbalance ratio as well as the small-sized class having unique characteristics.

E. DATASET CHALLENGES

It is necessary for a dataset to have enough diversity to build a robust model that can perform well in real world scenarios [49]. In cricket, difference in batting style, similarities across shot types, and environmental factors can introduce variation in a dataset. In our overall process of building the dataset, we ensured this variation to build a challenging and well-diversified dataset.

1) INTER CLASS SIMILARITY

Figure 7 illustrates examples of similarities between classes. In Figure 7a, the two shots, cover drive and straight drive, are played in very similar manners. Both the initial and final frames appear almost identical for each shot, while the middle frame exhibits the most noticeable variation in bat positioning. In the straight drive, the bat in the middle frame is relatively perpendicular to the ground, whereas, in the cover drive, it is held at a more angular position. A similar pattern is observed in the pull and hook shots in Figure 7b. Both target short-pitched deliveries towards the leg side of the field, with one played in the air and the other along the ground, resulting in slight differences in bat placement in middle frames.

2) INTRA CLASS SIMILARITY

Figure 8 shows examples of how shots within the same class can appear quite different. Both upper-cut shots in Figure 8a exhibit significant variation across frames. In addition, the camera angle in the bottom shot is from a more top-down perspective. For the scoop shots in Figure 8b, the variation is even more pronounced. In the top image, the player's body movement is more dynamic and upright, whereas in the bottom image, the player is crouching. This highlights the extent of variation the model needs to learn to accurately classify instances within the same class.

3) ENVIRONMENTAL FACTORS

Environmental and external factors, such as camera quality, time of day, and the cricket ground itself, can lead to noticeable visual differences across videos. These variations are evident across the sample videos presented in this work for various purposes. Figure 5 illustrates that each video has slightly different pitch conditions. Camera angles also vary, with some shots captured closer to the ground and others from higher viewpoints. Additionally, players wear a range of jerseys, from the plain white kits used in test matches to the colored uniforms seen in other formats.

F. SHOT DATASET AUGMENTATION

Several computer vision studies utilize various task-specific data preprocessing techniques to improve the reliability of the model and achieve higher accuracy [50], [51], [52]. Such preprocessing methods serve as effective alternatives to relying solely on large datasets to prevent overfitting [53]. Among these techniques, data augmentation is widely used to address overfitting by introducing additional variation into the training data [54]. Data augmentation increases the diversity of the training set by simple transformations such as

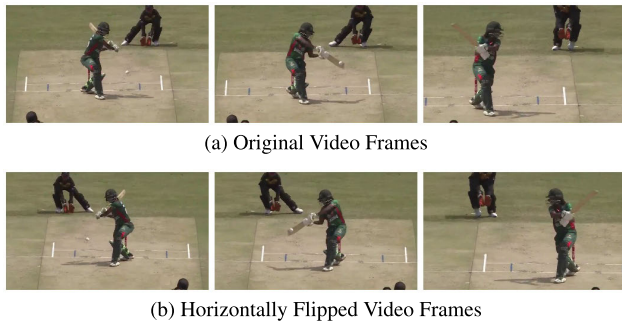


FIGURE 9. Illustration of horizontal flipping augmentation. In this video, the left-handed batter plays a square cut shot. The video is flipped to create a realistic variation for right-handed batter.

horizontal flipping, color space augmentations, and geometric modifications [55]. These augmentation techniques are required to be safe so that they preserve the original label post-transformation [56]. For example, rotations and flips are generally safe for cat versus dog classification, but not safe for digit recognition tasks such as 6 versus 9 [57].

Among various augmentation strategies, horizontal flipping is a safe and effective technique for cricket shot classification to create realistic shot variations commonly seen in actual matches. In cricket, batters are classified as right-handed or left-handed based on their dominant hand. Although their stances are horizontally opposite, the type of shot played is generally independent of handedness. The same shot can be executed by both types of batters with similar bat movements. In any dataset, a particular variation of a shot may only be present for one type of batter. Horizontal flipping helps artificially generate the missing variations, thereby improving the model's generalization. While other augmentation techniques are useful for addressing digital issues such as noise or low camera quality, we focused exclusively on horizontal flipping because it generates realistic physical variations that can naturally occur in cricket, effectively replicating actual gameplay scenarios.

It is crucial to perform augmentation after splitting the dataset into training, validation, and test sets. Some earlier works have applied flipping to artificially increase dataset size, which can create a misleading impression of dataset size and diversity. Moreover, if augmentations are applied before the split, similar synthetic samples may appear across different sets. This scenario is known as data leakage [58]. Data leakage can result in an overestimation of the model's true performance. To avoid these issues, we applied horizontal flipping augmentation solely to the training set after splitting the dataset. The dataset sizes reported in this work do not include these flipped samples. An example of horizontal flipping is illustrated in Figure III-F.

IV. METHODOLOGY: CRICSHOTNET

In this section, we provide a full overview of our proposed shot classification model, CricShotNet, along with a detailed description of each key layer. Figure 10 summarizes the

architecture of CricShotNet. The final part of this section discusses the experimental setup, the hyperparameter settings, and the metrics used to assess the model's performance.

A. CROPPING LAYER

In video footage from live broadcast camera perspectives, each frame contains elements irrelevant to shot classification. The type of cricket shot is determined by the striker's movement and the bat in their hand. Other events, like the ball's landing, trajectory, or bat contact, do not matter. Thus, cropping around the striker and bat preserves all context needed to identify the shot. The player type detection model used for data collection, discussed in Section III-B, was capable of classifying the striker. Hence, we reused the annotated dataset built for this purpose. Since the bat often received less focus due to its small size, occasionally causing parts of it to be cropped out, we trained an additional bat detection model on 750 images using YOLOv11m. We then combined the areas detected for the striker and bat by these two models to crop the region of interest for each video. The frames were then resized to 224×224 .

For each frame in a video, striker detection is performed first. If a striker is detected in a frame, a subsequent bat detection is performed for that frame. If the bat is detected, the bounding boxes of both objects are combined. If the bat is not detected, only the striker's bounding box is used. Even with the best model, it is possible that the bat is not detected because it can be hidden behind the striker or can appear very small due to the forward angle with the camera in certain bat swing positions. If the striker is not detected in a frame of a video, that frame is excluded from the cropped version of the video. This ensures that any irrelevant segments in a video that do not play a role in determining the shot type are excluded. These segments may appear in a video if the camera is switched to follow the trajectory of the ball after the striker has hit the ball. During annotation, videos with excessive such frames were marked as not a shot. After striker detection, the average number of frames across all videos decreased from 26.12 to 24.77, and the minimum dropped from 22 to 17. Since no video fell below 15 frames, this demonstrates both the high quality of annotation and the retention of sufficient temporal information. All cropped frames were resized to 224×224 , and 15 frames per video were uniformly sampled for GRU compatibility. The choice of 15 frames was made following extensive testing with varying numbers of frames, as well as visual inspection to ensure that sufficient information was retained. Figure 11 shows a sample output of a selected frame with an original size of 1280×720 , after cropping and resizing to 224×224 .

B. SEGMENTATION LAYER

The posture of the striker and the movement of the bat are the primary factors in classifying cricket shots. In formats other than Test matches, players wear different-colored jerseys, which can introduce noise. This highlights the need for a technique that can focus on certain visual information

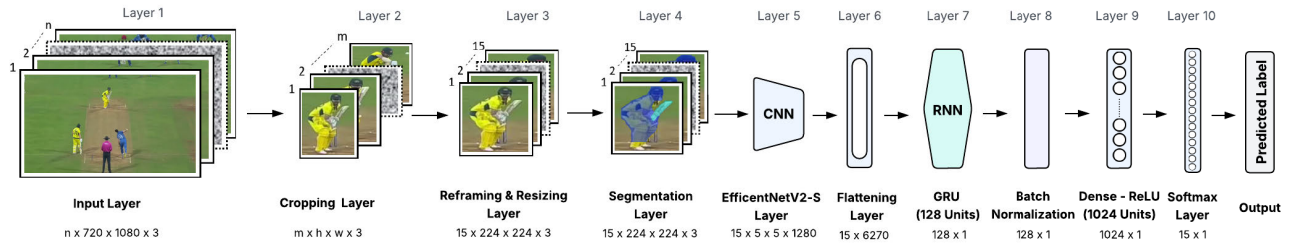


FIGURE 10. An illustration of overall model architecture of CricShotNet. The input is a 1-second video of 720×1280 size with variable frame counts. Each frame is cropped around the detected batter and bat, and frames without detection are discarded. To handle varying cropped regions and frame counts, 15 frames are uniformly sampled and resized to 224×224 . They pass through a segmentation layer that semi-transparently colors the batter and bat, then features are extracted using EfficientNetV2-S, flattened, and processed by a GRU block with 128 units. The output is classified via batch normalization, a dense ReLU layer (1024), and a final softmax block which produces the predicted label.

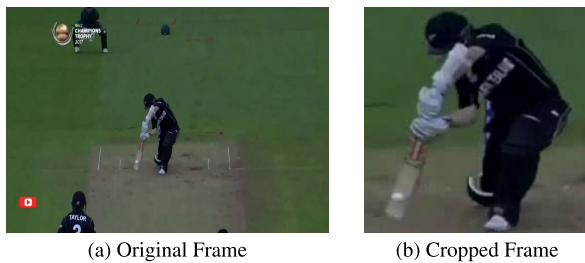


FIGURE 11. Illustration of cropping the area around the batter and the bat.

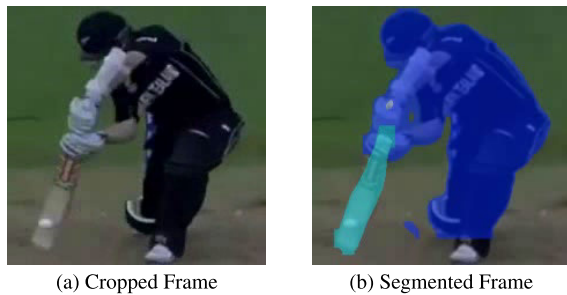


FIGURE 12. Illustration of instance segmentation and coloring the batter and the bat with semi-transparent colors.

while suppressing irrelevant variations. Enhancing the area of interest for a particular classification problem can significantly improve the performance of the model [59]. In our case, to focus on the most relevant regions, the bat and the striker, we use the instance segmentation technique, which can identify the precise shapes of objects or regions within an image [60]. Among alternative strategies, mask-only inputs may remove contextual cues like player posture and surrounding motion, while attention-based mechanisms may overlook the bat when it blends with the background due to its subtle. In contrast, our segmentation technique explicitly highlights the batter and bat through distinct coloring while preserving posture and contextual information, providing an effective way to incorporate domain knowledge into the model.

The segmentation model we used was fine-tuned on a baseline YOLO11x-seg architecture [6] with a curated dataset

comprising 674 images. The images used were taken from the frames of randomly selected cropped videos of our shot dataset. Two different classes, bat and striker, were annotated in each of the images. After segmentation, the predicted masks were overlaid with semi-transparent colors ($\alpha = 0.5$), highlighting the striker in blue and the bat in green. Semi-transparent color is chosen instead of full solid to avoid the loss of full information such as the position of the hands and rest of the body. The striker is colored first, followed by the bat, ensuring that the bat remains fully visible even when it overlaps the striker. Figure 12 shows an example of a cropped frame after segmentation.

C. DEEP FEATURE EXTRACTOR

Before capturing the relationship between frames, it is necessary to extract the deep features of individual frames for each video. Instead of training a feature extractor from scratch, using a transfer learning-based feature extractor significantly reduces computational complexity. For our CricShot10k dataset, EfficientNetV2-S [7] has demonstrated substantial performance superiority. EfficientNetV2 is a new family of faster and smaller models that outperforms others in both training speed and parameter efficiency, while also improving accuracy. The baseline model used is EfficientNetV2-S, while other models derived through the scaling method include EfficientNetV2-M and EfficientNetV2-L. For our proposed framework, EfficientNetV2-S, trained on the original ImageNet dataset [61], has proven to be sufficient to capture relevant information. The top layer, or classifier block, has been excluded. The input image size is set to $224 \times 224 \times 3$, and the entire model is wrapped in a TimeDistributed layer to apply the same processing to each frame of the video. The output of the model is then flattened to prepare it for the subsequent GRU layer.

In this work, EfficientNetV2-S demonstrated the best performance among a range of state-of-the-art transfer learning-based deep feature extractors. These included lightweight models such as MobileNetV2 [62], MobileNetV3-Large [63], and NASNet-Mobile [64], classic convolutional networks like VGG19 [65], ResNet50 and ResNet50V2

[66], dense architectures including DenseNet121 and DenseNet201 [67], inception-based networks such as InceptionV3 [68] and InceptionResNetV2 [69], and recent efficient models like EfficientNet-B3 and EfficientNetV2-B3 [7], [70], as well as ConvNeXt-Tiny [71] and Xception [72].

D. TEMPORAL FEATURE EXTRACTOR

Recurrent Neural Networks (RNNs) are widely used for modeling temporal patterns in time series data [73], but the vanishing gradient problem limits their ability to capture long-range dependencies. To mitigate this issue, two prominent RNN variants have been developed: Long-Short-Term Memory (LSTM) [74] and Gated Recurrent Unit (GRU) [75]. LSTM networks address this with memory cells and gates that retain information over extended periods, while GRU simplifies the architecture with fewer gates, making them more computationally efficient and less prone to overfitting. Variants such as Bidirectional LSTM (BiLSTM) and stacked LSTMs can process sequences in both directions, making them particularly suitable for complex video data [76].

The number of units in the hidden layers is an adjustable parameter in both LSTM and GRU. This parameter determines the network's capacity to store and process information. Choosing the appropriate number of units depends on the size, variability, and complexity of the dataset that the model needs to capture. A lower number of units may result in underfitting, whereas a higher number can increase computational cost and, in some cases, lead to overfitting if the data set is small or lacks diversity. Although any positive integer can be chosen, it is common practice to use powers of two (e.g., 32, 64, 128) due to their compatibility with hardware optimizations and to systematically explore variations in model capacity. After experimenting with different hidden units count for the LSTM, GRU and BiLSTM models, we found that the GRU model with 128 units achieved the best performance in the test set.

E. CLASSIFIER NETWORK

Instead of directly using the extracted features from the temporal feature extractor, to find the best-suited model for cricket shot classification, we tested a variety of combinations of dense and batch normalization blocks. Upon testing, we found that batch normalization followed by a dense layer with 1024 nodes performed the best. The output of the dense layer was then fed into a fully connected softmax classifier with 15 different classes. A Batch Normalization layer [77] was added between the output of the temporal feature extractor and the dense layer. A batch normalization block standardizes the inputs for the final layer for each mini-batch and stabilizes the learning process. Rectified Linear Unit (ReLU) [78] was used as the activation function of the dense layer. To classify each video clip into one of the fifteen shot types, the proposed model integrated a softmax activation function in the final fully connected layer with

fifteen output units. The value of each unit represents the probability that the input sample is in that class. Applying argmax on this layer provides the predicted class label.

F. EXPERIMENTAL SETUP

The experiments were performed on an NVIDIA RTX 3090 GPU with 10496 CUDA cores and 24 GB of GDDR6X VRAM. To read and process the videos during run-time, we have used the Decord² video reader. Decord accelerates training of deep neural networks on video data. Conventional video readers are often slow and inefficient, especially for large datasets and batch sizes. Decord improves efficiency by providing lightweight, hardware-accelerated video decoding and convenient slicing methods to enable direct video access during training.

The test sets were manually constructed, comprising 20% of the total videos. Care was taken to ensure that similar types of shots from the same batter do not overlap between the training and test sets. From the remaining videos, 20% were taken randomly for the validation test. Stratified splitting was applied to ensure that the class distribution is consistent across all splits and accurately reflects the overall dataset. A batch size of 8 was used, selected based on hardware limitations. Videos were randomly shuffled each epoch to introduce inter-batch variation. The shot classification models were trained for a maximum of 50 epochs. Early stopping was implemented with a patience value of 10. In our experiments, any improvement in validation loss smaller than 10^{-4} was considered a patient epoch. Across all runs, the models were able to converge in 30 epochs. Adam optimizer [79] was chosen for its adaptive learning rate, which allows faster convergence and better handling of sparse gradients. It is particularly useful for high-dimensional data, such as video sequences. The initial learning rate was set at 10^{-4} and was reduced by a factor of 0.1 if there were 4 consecutive patient epochs without significant improvement.

For detection and segmentation tasks, all training hyperparameters are summarized in Appendix C. All these hyperparameters follow the default settings of the YOLO training framework except for the patience and epoch count. The CNN models were initialized using the ImageNet [61] dataset. The weights in the GRU layer were initialized with the default Glorot uniform method [80] for the input-to-hidden weights, while the recurrent weights were initialized using orthogonal initialization, with biases set to zero. For all models, the trained data was saved after each epoch to allow for later loading and continued training. It is crucial to save the entire model state, including the weights and optimizer states, to ensure that training can resume seamlessly.

G. EVALUATION METRICS

This subsection outlines the evaluation metrics used for the shot classification task, as well as for the detection and

²<https://github.com/dmlc/decord>

segmentation models. All models are evaluated on a separate test set unseen during training and validation to ensure a fair assessment.

1) TOP-k ACCURACY

Accuracy measures the ratio of correctly predicted samples to total samples. Top-1 accuracy refers to standard accuracy, while top-k accuracy considers a prediction correct if the true label is among the top k predicted labels. Accuracy is unsuitable for detection or segmentation tasks, as it does not consider the spatial location of predicted regions.

$$\text{Top-k Accuracy} = \frac{\text{Correct Predictions in Top-k}}{\text{Total Number of Predictions}} \quad (1)$$

2) PRECISION

Precision is the ratio of true positives to the sum of true and false positives, indicating the accuracy of positive predictions. For binary classification, one class is positive and the other negative. In multiclass classification, precision is computed for each class considering that class as the positive class and all others as negative, and then their average is taken.

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}} \quad (2)$$

3) RECALL

Recall is the ratio of true positive to the sum of true positive and false negative predictions, indicating the ability of the model to correctly identify all relevant positive instances. Similarly to precision, recall in multiclass classification is calculated by taking the average of the recall of each individual class.

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \quad (3)$$

4) F1 SCORE

The F1 score is the harmonic mean of precision and recall, balancing both in a single metric. In multiclass settings, it is computed per class and then averaged. It is especially useful for uneven class distributions, as it penalizes extreme precision or recall values.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

5) AUC-ROC SCORE

The AUC-ROC score measures a model's ability to distinguish between classes by computing the area under the Receiver Operating Characteristic (ROC) curve, which plots the true positive rate against the false positive rate across different thresholds. Higher values indicate better class separation, with 1.0 denoting perfect discrimination and 0.5 indicating no discriminative ability. It is useful to evaluate the model's overall ability to differentiate between classes.

6) mAP50-95

mAP50–95 is an evaluation metric for object detection and segmentation models that computes mean Average Precision (mAP) across Intersection of Union (IoU) thresholds from 0.50 to 0.95 in steps of 0.05, where IoU measures the overlap between predicted and ground-truth regions. A simpler variant, mAP50, uses only an IoU of 0.50. We use these metrics to ensure robust evaluation of detection and localization performance.

7) PARAMETER COUNT

The parameter count denotes the total number of trainable variables, including weights and biases, and is an important metric for storage or computation-constrained applications such as IoT. Although cricket-based applications typically run on devices with sufficient computing power, models with fewer parameters are preferred when performance is comparable. Hence, parameter count is included as an evaluation metric.

V. RESULTS AND DISCUSSION

In this section, we present the results used to select different baseline architectures and identify the best-performing one for our proposed model. We also report experimental observations to demonstrate the effectiveness of our approach compared to prior work and conduct a qualitative error analysis.

A. PERFORMANCE ANALYSIS OF DIFFERENT BACKBONE CNN ARCHITECTURES

To identify the best backbone CNN model for deep feature extraction, we compared various metrics across 15 feature extractors, replacing only Layer 5 in the configuration of CricShotNet, as shown in Figure 10. Table 3 shows the results, with parameter counts reported for the CNN layer only. Although EfficientNetV2-B slightly outperforms in Top-2 accuracy, EfficientNetV2-S achieves the highest Top-1 accuracy at 89% and also the best Top-3 accuracy, precision, recall, and F1-score, with relatively low parameter count. Lightweight models like MobileNetV2 and MobileNetV3-Large reach Top-1 accuracies of 85% and 84%, respectively, making them suitable for less complex applications. In general, EfficientNetV2-S proved to be the best performing CNN architecture for our model. Furthermore, the consistently high Top-2 and Top-3 accuracies across most models (often exceeding 97% and 98%, respectively) suggest that even in cases of misclassification, the correct label frequently appears among the top predictions.

B. PERFORMANCE ANALYSIS OF DIFFERENT TEMPORAL FEATURE EXTRACTORS

The three variants of RNN models (LSTM, GRU, and BiLSTM) were evaluated with different hidden unit sizes, and their Top-1 accuracies are shown in Table 4. For consistency, BiLSTM models used equal hidden units in

TABLE 3. Performance comparison of different backbone CNN architectures with rest of the components of our proposed model fixed. (For every metric, bold = highest and underlined = second highest).

Model	Params (M)	Top-k Accuracy (%)			Precision	Recall	F1-Score
		Top-1	Top-2	Top-3			
EfficientNetV2-S [7]	20.33	89.09	<u>97.22</u>	99.25	0.8846	0.8881	0.8847
Xception [72]	20.86	<u>88.64</u>	97.17	98.51	<u>0.8722</u>	0.8690	0.8655
EfficientNetV2-B3 [7]	12.93	88.39	97.32	98.81	0.8718	<u>0.8770</u>	<u>0.8727</u>
ConvNeXt-Tiny [71]	27.82	87.20	<u>97.22</u>	<u>99.15</u>	0.8670	0.8568	0.8596
MobileNetV2 [62]	2.26	85.02	95.48	97.66	0.8413	0.8515	0.8432
EfficientNet-B3 [70]	28.95	84.47	95.83	98.21	0.8300	0.8314	0.8233
MobileNetV3-Large [63]	2.96	84.03	95.93	98.36	0.8423	0.8192	0.8274
ResNet50V2 [66]	23.56	84.52	94.99	98.21	0.8320	0.8430	0.8357
InceptionResNetV2 [69]	54.34	84.38	94.64	97.42	0.8139	0.8289	0.8161
NASNet-Mobile [64]	4.27	83.63	94.79	97.66	0.8391	0.8205	0.8199
ResNet50 [66]	23.59	79.76	93.15	96.97	0.7839	0.7976	0.7887
DenseNet201 [67]	18.32	75.15	91.17	96.47	0.7262	0.7178	0.7036
VGG19 [65]	20.02	75.64	90.18	95.88	0.7439	0.7015	0.7109
DenseNet121 [67]	7.03	74.50	90.37	95.23	0.7265	0.7335	0.7150
InceptionV3 [68]	21.80	68.84	83.13	91.26	0.7145	0.6324	0.6183

TABLE 4. Performance comparison of different architectures for RNN Layer with rest of the components of our proposed model fixed.

Model	Units	Params (M)	Acc. (%)
LSTM	64	16.07	85.42
	128	32.17	84.97
	256	64.48	83.76
GRU	64	12.05	85.52
	128	24.13	89.09
	256	48.36	86.06
BiLSTM	32+32	16.06	83.38
	64+64	32.15	87.65
	128+128	64.36	85.32

both directions. The parameter counts reported in table refer only to layer 7 in Figure 10. Different CNN backbones produce feature maps of varying dimensions that affect the RNN input size and parameter count. In this case, the CNN feature extractor was fixed to the best performing one, EfficientNetV2-S, with all other layers unchanged. The results in the table show that GRU with 128 hidden units achieves the highest Top-1 accuracy while requiring fewer parameters than larger configurations. Increasing hidden units to 256 reduced accuracy, likely due to overfitting. GRU models outperformed LSTM models. Although BiLSTM came close to outperforming in some cases, GRU was sufficient here because forward bat movement is most critical, with prior frame positions having minimal impact.

C. ABLATION STUDY

For any video classification task, a deep feature extractor and a temporal feature extractor are essential layers. In addition, we have introduced two initial layers: cropping and segmentation specialized for cricket shot classification. We have also applied horizontal flipping as part of the augmentation

TABLE 5. Ablation study of dedicated components of our proposed model, CricShotNet.

Cropping	Segmentation	Augmentation	Acc. (%)
×	×	×	72.03
×	×	✓	75.50
×	✓	×	79.45
×	✓	✓	80.20
✓	×	×	83.39
✓	×	✓	85.76
✓	✓	×	87.60
✓	✓	✓	89.09

process to generalize shots for both handed batters. Table 5 presents the ablation study of these three cricket-specific components. The results show that augmentation in any form improves performance. Furthermore, excluding either cropping or segmentation reduces performance. While keeping augmentation consistent, we observe that using both cropping and segmentation yields an accuracy of 89.09%, whereas using only cropping results in 85.76%. We also examined segmentation without cropping: when videos were not cropped but segmentation was applied, accuracy reached 80.20%, compared to 75.50% without segmentation. Overall, these results highlight the clear benefit of cropping the frames and then adding semi-transparent segmentation overlays in the pre-processing pipeline of CricShotNet.

D. CLASS WISE ANALYSIS

The precision, recall, F1-score, and AUC-ROC score of each class for our best proposed model are shown in Table 6. These results reveal important details about data imbalance and class complexity. The lowest-performing class in terms of these metrics is the late cut, with a recall of 0.6786 and an F1-score of 0.7451. This class has a total of 325 samples,

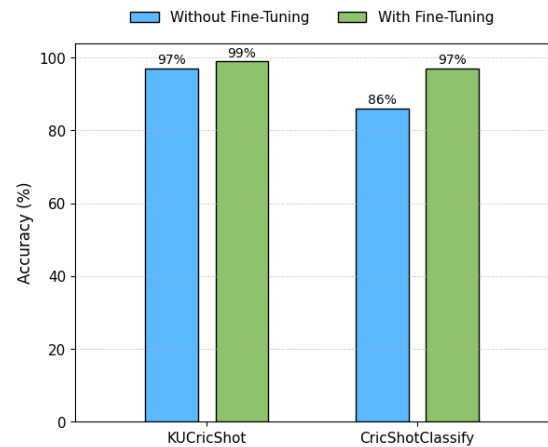
TABLE 6. Performance across different classes for our proposed model, CricShotNet.

Class	Precision	Recall	F1-score	AUC-ROC
Cover Drive	0.8155	0.9320	0.8698	0.9968
Defensive	0.9280	0.9206	0.9243	0.9982
Down The Wicket	0.9758	0.8703	0.9200	0.9972
Flick	0.9568	0.9725	0.9646	0.9990
Hook	0.8087	0.8774	0.8416	0.9937
Late Cut	0.8261	0.6786	0.7451	0.9958
Lofted Legside	0.8850	0.8734	0.8791	0.9922
Lofted Offside	0.8182	0.8507	0.8341	0.9881
Pull	0.8828	0.7847	0.8309	0.9882
Reverse Sweep	0.8727	0.9796	0.9231	0.9992
Scoop	0.9828	1.0000	0.9913	1.0000
Square Cut	0.9091	0.9179	0.9135	0.9967
Straight Drive	0.8636	0.8636	0.8636	0.9965
Sweep	0.9609	0.9297	0.9451	0.9986
Upper Cut	0.7833	0.8704	0.8246	0.9946
Average	0.8846	0.8881	0.8847	0.9956

which is on the lower end of support. In addition to the lower support, the main reason for these low scores is the complexity of the data in this class. The late cut shot can be played in many different ways, and several variations closely resemble other classes, such as Upper Cut and Square Cut. Some variations of the shot are quite unique and may have only a few instances in the entire dataset. However, the class with the lowest number of samples (Reverse Sweep, 257 samples) achieves a high precision, recall, F1-score, and AUC of 0.8727, 0.9796, 0.9231, and 0.9992, respectively. The second-lowest sample class (Scoop, 276 samples) achieves even higher scores: 0.9828, 1.0000, 0.9913, and 1.0000. These results are in some cases superior to those of classes with far more samples, indicating that the dataset's imbalance does not hinder the model's robustness or generalization. Instead, performance is more influenced by the visual similarity between certain types of shots. The way some of these shots are played is highly distinctive, with virtually no resemblance to other classes. High ROC-AUC values across all classes (above 0.9880) confirm strong discriminative power between classes. Overall, most classes score above 0.85 in most metrics, demonstrating the success of the model in ensuring robust generalization.

E. COMPARISON WITH OTHER EXPERIMENTS

Table 7 presents a comparison of our dataset, CricShot10k, with existing datasets on cricket shot videos. Our work covers more classes and uses a dataset several times larger than previous studies. It also addresses the dataset quality limitations of previous works discussed in Section II-D. The proposed model, CricShotNet, achieved an accuracy of 89%, which aligns with performance typically observed in video classification tasks using datasets of comparable scale and variation. Since the datasets differ in structure and complexity, a direct model-to-model comparison is not entirely appropriate. Therefore, we also trained our

**FIGURE 13.** Cross-dataset performance of CricShotNet on existing cricket video datasets. *Without fine-tuning* refers to testing CricShotNet trained solely on CricShot10k without exposure to the target dataset, whereas *with fine-tuning* indicates additional training on the target dataset before evaluation.

model on identical datasets to demonstrate its relative performance, as shown in Table 7. For all these external datasets, our model consistently outperformed the models of the corresponding authors. Furthermore, when tested on our dataset, CricShotNet surpassed all existing approaches. The substantially lower accuracies of these other methods highlight their limitations when applied to a more challenging and diverse dataset.

To evaluate the cross-dataset performance of CricShotNet, trained exclusively on CricShot10k, we also tested it on the KUCricShot and CricShotClassify datasets without any fine-tuning. Figure 13 illustrates CricShotNet's performance on these datasets, which were independently prepared by other authors. All previously reported accuracies of CricShotNet were obtained on fully unseen test sets from CricShot10k. Extending the evaluation to entirely independent datasets further strengthens the robustness claim of CricShotNet. On the KUCricShot dataset, CricShotNet achieves a remarkably high accuracy of 97% despite differences in collection methodology and annotation, likely aided by the low class count of KUCricShot. Fine-tuning on this dataset further improves performance to near-perfect 99%. For CricShotClassify, the initial accuracy without fine-tuning is 86%, which can be attributed to longer video lengths and some mislabeling, as discussed in Section II. Nevertheless, this demonstrates that CricShotNet performs comparably to state-of-the-art video classification models. Fine-tuning on CricShotClassify boosts accuracy to 97%, surpassing that reported by the original authors. This demonstrates the strong generalizability of CricShotNet, even when tested on previously unseen or independent datasets.

F. ERROR ANALYSIS

Figure 14 shows the confusion matrix for our best proposed model. It reveals that lofted legside and pull were among the most misclassified shots. Notably, 8% of lofted legside

TABLE 7. Comparison of datasets and performance with existing works on cricket shot classification from videos.

Dataset	Class	Size	Compared Authors	Authors' Model Type	Accuracy of Authors' Model (%)	Accuracy of our CricShotNet (%)
KUCricShot [4]	4	1,278	Hoque <i>et al.</i> [4]	LRCN	73	98.86
CricShotClassify [5]	10	1,867	Sen <i>et al.</i> [5]	VGG16-GRU	93	97.10
CricShot10k (ours)	15	10,086	Hoque <i>et al.</i> [4]	LRCN	41	89.09
			Sen <i>et al.</i> [5]	VGG16-GRU	54	
			Rao <i>et al.</i> [29]	MediaPipe-GRU	32	

True Class	Flick	177	0	3	0	0	0	1	0	0	0	1	0	0	0	0	0
	Late Cut	0	38	4	0	1	0	4	8	0	1	0	0	0	0	0	0
	Defensive	1	3	116	0	0	0	1	0	0	0	0	5	0	0	0	0
	Lofted Legside	1	0	0	200	1	0	0	0	3	1	0	0	3	20	0	0
	Down The Wicket	1	0	0	8	161	0	5	0	1	0	0	1	2	6	0	0
	Scoop	0	0	0	0	0	57	0	0	0	0	0	0	0	0	0	0
	Cover Drive	0	1	0	0	0	0	137	0	0	0	0	5	0	4	0	0
	Square Cut	0	4	0	0	0	0	3	190	0	0	6	0	0	4	0	0
	Pull	2	0	0	1	0	0	0	3	113	19	2	0	1	3	0	0
	Hook	0	0	0	2	0	0	0	0	7	93	4	0	0	0	0	0
	Upper Cut	0	0	0	0	0	0	0	5	1	1	47	0	0	0	0	0
	Straight Drive	3	0	2	0	0	0	5	1	0	0	0	76	0	1	0	0
	Sweep	0	0	0	2	0	0	1	0	3	0	0	0	172	0	7	0
	Lofted Offside	0	0	0	13	2	0	11	2	0	0	0	1	1	171	0	0
	Reverse Sweep	0	0	0	0	0	0	1	0	0	0	0	0	0	0	48	0
	Flick																
	Late Cut																
	Defensive																
	Lofted Legside																
	Down The Wicket																
	Scoop																
	Cover Drive																
	Square Cut																
	Pull																
	Hook																
	Upper Cut																
	Straight Drive																
	Sweep																
	Lofted Offside																
	Reverse Sweep																
	Predicted Class																

FIGURE 14. Confusion matrix.**(a)** Sweep shot misclassified as lofted legside (top) and a similar looking lofted legside (down)**(b)** Defensive shot misclassified as straight drive (top) and a similar looking straight drive (down)**FIGURE 15.** Error analysis with similar looking samples.

shots were misclassified as lofted offside. The highest misclassification between any two shots occurred with pull and hook, where 13% of pull shots were misclassified as

TABLE 8. Performance of different object detection and segmentation models used across various parts of our work.

Model	Support	Type	Precision	Recall	mAP50	mAP50-95
Player Detection	602	Box	0.9498	0.9310	0.9508	0.7180
Ball Detection	1,400	Box	0.9289	0.9013	0.9550	0.5083
Bat Detection	750	Box	0.9726	0.8493	0.8976	0.5736
Segmentation	674	Box	0.9658	0.8793	0.9133	0.7291
		Mask	0.9662	0.8879	0.9053	0.6556

**(a)** Early capture of the first frame**(b)** Late capture of the first frame**(c)** Insufficient context in the fixed one-second window**FIGURE 16.** Examples of discarded videos that are labeled as not a shot for different reasons.

hook. Both of these pairs are visually quite similar, which explains the misclassifications. The shot with the fewest misclassifications is scoop. Interestingly, not a single scoop shot out of 57 was classified as any other shot, and only a single reverse sweep shot was misclassified as scoop. This demonstrates how visually distinct the scoop shot is and how accurately the model was able to predict it. Other notable shots with limited misclassifications are reverse sweep and flick, with 97.95% and 97.25% of them correctly predicted, respectively.

In Figure 15, we investigate the potential reasons behind these misclassifications. The confusion with lofted legside mainly occurred with lofted offside, sweep, and pull shots. To illustrate these errors, we present a misclassification

TABLE 9. Evaluation of shot-splitting accuracy of the automated method on selected videos.

Video No	Match Details	True Shot Count	Trimmed by Model	Validly Trimmed Shot Count	Missed by Model
24	Australia vs Sri-Lanka, ODI WC 2007 Final	31	28	27	4
61	Bangladesh vs Scotland, Women's T20 WC 2024 Final	19	18	18	1
142	India vs Australia, Under 19 Men's WC 2024 Final	32	31	29	3
235	England vs India, Day 1, 1st Test, 2021	52	53	50	2
450	Gujarat Giants vs Mumbai Indians, WPL 2023	38	37	34	4

TABLE 10. Training hyperparameter configurations for the YOLO models.

Purpose	Trained On	Image size	Epochs	Patience
Player Detection	YOLOv11m	720	100	10
Ball Detection	YOLOv11m	720	200	30
Bat Detection	YOLOv11m	720	100	10
Segmentation	YOLOv11x-seg	224	200	30

between sweep and lofted legside in Figure 15a. In this sweep shot, the batter places his back foot on the ground slightly later than in most sweep shots, leading the model to ignore the wider bat swing in the middle frame and instead classify it as a lofted legside. On the other hand, the defensive shot was one of the least misclassified shots, but most of its few errors were in favor of the straight drive. Only five defensive shots were misclassified as straight drive. Using Figure 15b, we explore one example of this contrasting scenario. In this video, the batter attempts a back-foot defensive shot, where the bat is held slightly higher than in a typical front-foot defensive shot. This subtle variation caused the model to mistake the action as the batter lifting the bat to generate momentum for a straight drive. Overall, while a few shots were misclassified by CricShotNet, these errors are interpretable, suggesting clear directions for future work to achieve targeted improvements.

G. MISCELLANEOUS MODEL PERFORMANCE

Table 8 presents the performance of various object detection and segmentation models evaluated in our study across different components. In each case, even with relatively small datasets, the models achieved standard performance for their respective tasks. Notably, the ball detection model performed comparably well, despite the ball being a relatively small object, with its larger dataset contributing to this performance.

H. RESEARCH ACHIEVEMENTS

We extracted one-second-long shot videos from a total of 536 full-length match videos, which ranged from 5 to 35 minutes in duration, with an average length of approximately 12 minutes. This amounts to a total of 6,432 minutes, or around 107 hours of footage. To generate ideal one-second shot clips manually, an annotator would have to watch all 107 hours, and even optimistically, the process of identifying the start and end frames for each clip, along with other necessary steps, would require approximately five times

more time. Hence, we effectively saved around 500 hours of manual effort through automation.

The final dataset consists of 10,086 one-second clips, totaling 10,086 seconds (approximately 168 minutes, or 2.8 hours). In other words, we converted 107 hours of raw video into 2.8 hours of curated, high-quality shot clips through automation. We also note that by reducing the task to one-second clips, the annotation process becomes similar to standard text or image annotation tasks, where the unit of labeling is small enough for efficient human evaluation. By employing three annotators with strong cricket knowledge and requiring consensus among them for each clip, we ensured high-quality, reliable annotations across the dataset. We believe that this initial annotation effort can serve as a foundation for future work, enabling more efficient data scraping and potentially reducing or even eliminating the need for manual annotation when generating large cricket shot datasets. Additionally, such a highly accurate dataset can serve as a valuable resource for automated commentary generation and other cricket-related applications.

VI. CONCLUSION

In this paper, we present CricShot10k, the largest and most diverse cricket shot dataset to date, comprising 10,086 videos across 15 classes. To streamline dataset creation, we propose an automated method that segments full-length match videos into concise clips, each containing a single shot. This method enables efficient construction of large-scale datasets. We provide a detailed analysis of CricShot10k, underscoring its challenges and variations to demonstrate the robustness of the dataset. Our cricket shot classification model, CricShotNet, enhanced with two additional layers: cropping and segmentation, significantly outperforms existing approaches. Through extensive experiments, we identify EfficientNetV2S and a GRU with 128 hidden units as the best choices for subsequent layers. The results validate both the reliability and difficulty of our dataset, as well as the strength of our proposed classification framework.

Future studies can explore expanding the dataset by adding new class labels, such as leave, leg glance, and backfoot punch, subdividing existing classes like defensive into front-foot and back-foot categories, and increasing the number of samples within each class. In our case, we used videos of fixed duration. Although this did not affect dataset quality thanks to careful annotation, accurately detecting the exact time frame of a shot would help overcome this

limitation. This can be addressed by developing an activity recognition model for cricket to automatically identify when a batter plays the shots in a video. Our shot dataset could serve as a solid starting point for such efforts. The task of shot classification can be further explored using different transformer models, such as Video Swin and TimeSformer, advanced data augmentation techniques like temporal jittering and color perturbation, and more domain-specific architectures, such as tracking the motion of the bat. Finally, this work can be extended to detect other types of cricket events on the field, contributing towards the development of an automated cricket commentary generator.

APPENDIX A

AUTOMATED DATA COLLECTION METHOD

To divide a long cricket match recording into multiple one-second shot clips, we begin by processing the full length video. Each frame is passed through a YOLOv11-based detection pipeline to identify the presence of the batter, bowler, and ball. To ensure stable and noise-resistant detection, we maintain four parallel sliding-window queues of length 10: one for frames and three corresponding to the batter, bowler, and ball detections. For each incoming frame, the output of the YOLOv11 detector is encoded as a binary value, where the detection of the respective element (batter, bowler, or ball) is stored as 1, and the absence of a detection is stored as 0. These values are appended to their respective sliding-window queues. The queues operate in a first-in–first-out manner, so once the window size reaches 10, the oldest element is removed upon insertion of a new one.

As discussed in the main text, we do not consider a shot has begun simply when all three entities are detected in a single frame. Instead, we impose a set of temporal consistency constraints over the 10-frame sliding window to improve robustness. A frame is considered the starting point of a shot only if the following three conditions are simultaneously satisfied within its corresponding 10-frame window.

- 1) The bowler appears in at least one of the first five frames of the window.
- 2) The batter appears in at least five of the ten frames.
- 3) The ball appears in at least five of the ten frames.

These thresholds were chosen after extensive experiments, comparing the number of validly trimmed and missed shots across a set of videos for different values during the initial phase of the work. The bowler constraint is restricted to the first five frames because, after releasing the ball, the camera often zooms in to track the ball trajectory, causing the bowler to exit the frame. Since the moment of ball release is critical for determining the start of a shot, ensuring the bowler's presence in the early portion of the window provides a more reliable cue. The benefit of this constraint is illustrated in Figure 4.

To illustrate the operation of the method, consider the following example state of the three detection arrays.

Frame	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}
Bowler	0	1	1	0	0	0	0	0	0	0
Batter	0	1	1	1	0	1	1	1	1	1
Ball	0	0	0	1	0	0	0	1	1	1

In this example, the bowler is present in two (f_4 and f_5) of the first five frames, and the batter is present in eight of the ten frames, satisfying both conditions. However, the ball appears in only four of the ten frames. Therefore, f_3 is not considered an ideal starting frame for the shot.

Through extensive experimentation, we observed that most detection errors occur due to the ball. Being a small object, the ball can often be undetected even by human observers. By contrast, small circular objects can sometimes be mistakenly identified as the ball, even with state-of-the-art models. After testing various parameter settings, we found that requiring the ball to appear in at least five out of ten frames provides a reasonable balance: it minimizes false detections while allowing for occasional missed frames where the ball is obscured. This parameter choice avoids reducing the amount of extracted clips significantly and also prevents the erroneous extraction of clips that are not actual shots, thereby saving annotation effort.

Now, consider the arrival of a new frame f_{13} in which both the batter and the ball are detected. The updated state of the arrays is as follows.

Frame	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}
Bowler	1	1	0	0	0	0	0	0	0	0
Batter	1	1	1	0	1	1	1	1	1	1
Ball	0	0	1	0	0	0	1	1	1	1

At this stage, all conditions are satisfied: the first two frames contain the bowler, the batter appears in nine frames, and the ball is present in five frames. Therefore, f_4 can be considered the ideal starting frame for capturing a one-second shot clip. Given that the video has a frame rate of 30 fps, the shot clip spans from f_4 to f_{33} . To avoid overlapping shots, the sliding-window arrays are then reset, and the process of pushing, popping and counting frames begins anew from f_{34} in search of the next shot.

APPENDIX B

LIMITATIONS OF THE AUTOMATED METHOD

The fixed one-second clip duration was chosen as a practical compromise after extensive testing on matches with diverse characteristics. After collecting a total of 14,369 one-second clips, 2,443 clips had to be labeled as non-shots (Table 1). These non-shots correspond to cases where the bowler, batter, or ball was detected in an inappropriate context, as illustrated in Figure 6. Although such cases are rare, some occurred because the first frame was captured either too early or too late. The rarest cases arose when the first frame was correctly

identified, but the batter did not complete their action within the one-second window.

During the annotation process, we took great care to exclude any videos that lacked sufficient temporal context for capturing nuanced movements. Figure 16 illustrates examples of three videos discarded for distinct reasons. For each video, the first, middle, and last frames are shown to highlight the missing contextual information. In Figure 16a, the first frame was captured too early, likely due to a misdetection of the ball. As a result, even by the middle frame, the bowler had not yet released the ball, and the last frame was recorded well before the batter completed their swing. In contrast, Figure 16b shows a late capture of the first frame, where the batter is already halfway through their swing. This can be confirmed by the last frame, in which the camera zooms out to follow the ball. Importantly, these cases are not caused by the fixed one-second clip duration but by early or late detection of the first frame. Figure 16c, however, demonstrates the rare limitation imposed by the one-second clip duration. Although the first frame was captured at an appropriate moment, the ball was delivered unusually slowly, preventing the batter from completing their action within one second. Such instances are extremely uncommon, with fewer than 100 occurrences among 14,369 collected clips.

These misaligned clips constitute only 17% of all trimmed videos, a negligible fraction when weighed against the substantial reduction in manual annotation effort and the efficiency gains in dataset construction. By explicitly acknowledging and quantifying these rare cases, we provide a transparent account of the method's limitations while highlighting the robustness and practicality of our automated clipping method. It was ensured during the annotation process that the quality of the dataset was not compromised despite these limitations. Future work may explore methods to further minimize such occurrences. Overall, our dataset can serve as a strong reference for future work on cricket. Note that, the dataset is intended solely for non-commercial research purposes and the copyright of the original videos remains with the respective rights holders.

An additional consideration is whether every shot in the highlights videos is correctly split. A shot can be missed if the object detection model fails to detect a key element: the ball, the bowler, or the striker. While manually verifying all 536 match videos is not feasible, we selected five representative videos with diverse characteristics to estimate missed shots. For each video, we manually counted the true number of shots, compared them with the model-trimmed shots, and distinguished between valid and invalid trims. The number of missed shots per video was calculated as the difference between valid trims and the manually counted true shots. These metrics are summarized in Table 9. As shown, most shots were validly trimmed, with the lowest counts being 1 out of 19 for video 61 and 2 out of 52 for video 235, indicating that missed or invalid trims are rare.

APPENDIX C

HYPERPARAMETERS SUMMARY

Table 10 summarizes the training configurations of all YOLO models used in this work. For models with higher complexity, such as the ball detection model, larger values for epochs and patience were employed. In most cases, the detection models converged within 100 epochs, while the segmentation model converged within 200 epochs. During evaluation and inference, the maximum number of detections was set according to the task: for ball detection, at most one detection was allowed; for segmentation, which aimed to segment both the bat and the player, the maximum was set to two. No limit was imposed for the player detection model, which is designed to detect any number of players, including both the striker and the bowler. All other training and evaluation parameters were set to their default values. We have added these datasets to the repository to ensure reproducibility.

ACKNOWLEDGMENT

The authors would like to thank Md. Shohanur Rahman, TiCON System Ltd., Dhaka, Bangladesh, and Abdullah As-Sadeed, Programmer at AlmaLinux OS Foundation for their valuable time and advice to make this study possible.

CONFLICT OF INTEREST

The authors declare that there is no conflict of interest.

REFERENCES

- [1] (2025). *The World's Most Watched Sports*. Accessed: Dec. 12, 2025. [Online]. Available: <https://sportforbusiness.com/the-worlds-most-watched-sports/>
- [2] M. F. A. Foyyal, M. S. Islam, A. Karim, and N. Neehal, "Shot-net: A convolutional neural network for classifying different cricket shots," in *Proc. 2nd Int. Conf. Recent Trends Image Process. Pattern Recognit.*, Solapur, India. Cham, Switzerland: Springer, Dec. 2019, pp. 111–120.
- [3] J. B. Fernandes, P. S. Ram, P. M. V. Yadav, and K. P. Kumar, "Cricket shot detection using 2D CNN," in *Proc. 7th Int. Conf. Intell. Comput. Control Syst. (ICICCS)*, May 2023, pp. 608–612.
- [4] S. W. Hoque, M. A. Habib, A. Shome, and S. Sakib, "KUCricShot: A dataset for developing detection model of cricket shots," in *Proc. 26th Int. Conf. Comput. Inf. Technol. (ICCIT)*, Dec. 2023, pp. 1–6.
- [5] A. Sen, K. Deb, P. K. Dhar, and T. Koshiba, "CricShotClassify: An approach to classifying batting shots from cricket videos using a convolutional neural network and gated recurrent unit," *Sensors*, vol. 21, no. 8, p. 2846, Apr. 2021.
- [6] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," 2024, *arXiv:2410.17725*.
- [7] M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 10096–10106.
- [8] M. Lames and T. McGarry, "On the search for reliable performance indicators in game sports," *Int. J. Perform. Anal. Sport*, vol. 7, no. 1, pp. 62–79, Jan. 2007.
- [9] T. McGarry, "Applied and theoretical perspectives of performance analysis in sport: Scientific issues and challenges," *Int. J. Perform. Anal. Sport*, vol. 9, no. 1, pp. 128–140, Apr. 2009.
- [10] S. E. Moritz, D. L. Feltz, K. R. Fahrbach, and D. E. Mack, "The relation of self-efficacy measures to sport performance: A meta-analytic review," *Res. Quart. for Exercise Sport*, vol. 71, no. 3, pp. 280–294, Sep. 2000.
- [11] A. McCabe and J. Trevathan, "Artificial intelligence in sports prediction," in *Proc. 5th Int. Conf. Inf. Technology: New Generat.*, Apr. 2008, pp. 1194–1197.
- [12] B. Min, J. Kim, C. Choe, H. Eom, and R. I. (Bob) McKay, "A compound framework for sports results prediction: A football case study," *Knowl.-Based Syst.*, vol. 21, no. 7, pp. 551–562, Oct. 2008.

- [13] A. Maszczyk, A. Gołaś, P. Pietraszewski, R. Roczniok, A. Zajac, and A. Stanula, "Application of neural and regression models in sports results prediction," *Proc. Social Behav. Sci.*, vol. 117, pp. 482–487, Mar. 2014.
- [14] A. Joseph, N. E. Fenton, and M. Neil, "Predicting football results using Bayesian nets and other machine learning techniques," *Knowl.-Based Syst.*, vol. 19, no. 7, pp. 544–553, Nov. 2006.
- [15] Y. Yoon, H. Hwang, Y. Choi, M. Joo, H. Oh, I. Park, K.-H. Lee, and J.-H. Hwang, "Analyzing basketball movements and pass relationships using realtime object tracking techniques based on deep learning," *IEEE Access*, vol. 7, pp. 56564–56576, 2019.
- [16] G. Jin, "Player target tracking and detection in football game video using edge computing and deep learning," *J. Supercomput.*, vol. 78, no. 7, pp. 9475–9491, May 2022.
- [17] Y.-C. Li, C.-T. Chang, C.-C. Cheng, and Y.-L. Huang, "Baseball Swing pose estimation using OpenPose," in *Proc. IEEE Int. Conf. Robot. Autom. Artif. Intell. (RAAI)*, Apr. 2021, pp. 6–9.
- [18] A. Ganser, B. Hollaus, and S. Stabinger, "Classification of tennis shots with a neural network approach," *Sensors*, vol. 21, no. 17, p. 5703, Aug. 2021.
- [19] M. Goyani, S. Dutta, G. Gohil, and S. Naik, "Wicket fall concept mining from cricket video using A-priori algorithm," *Int. J. Multimedia Its Appl.*, vol. 3, no. 1, pp. 111–120, Feb. 2011.
- [20] M. Goyani, S. K. Dutta, and P. Raj, "Key frame detection based semantic event detection and classification using heirarchical approach for cricket sport video indexing," in *Proc. Int. Conf. Comput. Sci. Inf. Technol.*, 2010, pp. 388–397.
- [21] S. Nandyal and S. L. Kattimani, "Bird swarm optimization-based stacked autoencoder deep learning for umpire detection and classification," *Scalable Computing: Pract. Exper.*, vol. 21, no. 2, pp. 173–188, Jun. 2020.
- [22] K. R. Raval and M. M. Goyani, "Shot segmentation and replay detection for cricket video summarization," in *Proc. Int. Conf. Comput. Intell. Sustain. Eng. Solutions (CISES)*, Apr. 2023, pp. 933–938.
- [23] H. Sattar, M. S. Umar, E. Ijaz, and M. U. Arshad, "Multi-modal architecture for cricket highlights generation: Using computer vision and large language model," in *Proc. 17th Int. Conf. Open Source Syst. Technol. (ICOSST)*, Dec. 2023, pp. 1–6.
- [24] K. R. Raval and M. M. Goyani, "A survey on event detection based video summarization for cricket," *Multimedia Tools Appl.*, vol. 81, no. 20, pp. 29253–29281, Aug. 2022.
- [25] A. Suryavanshi, V. Kukreja, S. Mehta, A. Garg, and P. Tiwari, "Innovative innings: Exploring cricket shot categories with CNN-SVM methodologies," in *Proc. IEEE Int. Conf. Inf. Technol., Electron. Intell. Commun. Syst. (ICITEICS)*, Jun. 2024, pp. 1–6.
- [26] A. Dey and S. Biswas, "Shot-ViT: Cricket batting shots classification with vision transformer network," *Int. J. Eng.*, vol. 37, no. 12, pp. 2463–2472, 2024.
- [27] D. Karmaker, A. Z. M. E. Chowdhury, M. S. U. Miah, M. A. Imran, and M. H. Rahman, "Cricket shot classification using motion vector," in *Proc. 2nd Int. Conf. Comput. Technol. Inf. Manage. (ICCTIM)*, Apr. 2015, pp. 125–129.
- [28] J. Donahue, L. A. Hendricks, S. Guadarrama, M. Rohrbach, S. Venugopalan, T. Darrell, and K. Saenko, "Long-term recurrent convolutional networks for visual recognition and description," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 2625–2634.
- [29] C. G. Rao, N. S. H. Varma, P. Mazumder, and P. Ramasamy, "Cricket shot classification from video using body pose landmarks," in *Proc. 5th Int. Conf. Adv. Electr., Comput., Commun. Sustain. Technol. (ICAECT)*, Jan. 2025, pp. 1–8.
- [30] I. Bhat, T. Sridhar, M. A. Yajur, and S. R. Upadhyaya, "Building a video dataset for cricket shot analysis," in *Proc. Int. Conf. Netw., Multimedia Inf. Technol. (NMITCON)*, Sep. 2023, pp. 1–6.
- [31] L. Wang, Y. Xiong, Z. Wang, Y. Qiao, D. Lin, X. Tang, and L. Van Gool, "Temporal segment networks for action recognition in videos," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 11, pp. 2740–2755, Nov. 2019.
- [32] (2025). *Batting (cricket)*. Accessed: Dec. 12, 2025. [Online]. Available: [https://en.wikipedia.org/wiki/Batting\(cricket\)](https://en.wikipedia.org/wiki/Batting(cricket))
- [33] J. C. Chang, S. Amershi, and E. Kamar, "Revolt: Collaborative crowd-sourcing for labeling machine learning datasets," in *Proc. CHI Conf. Human Factors Comput. Syst.*, May 2017, pp. 2334–2346.
- [34] R. M. Alamgir, A. A. Shuvro, M. Al Mushabbir, M. A. Raiyan, N. J. Rani, M. M. Rahman, M. H. Kabir, and S. Ahmed, "Performance analysis of YOLO-based architectures for vehicle detection from traffic images in Bangladesh," in *Proc. 25th Int. Conf. Comput. Inf. Technol. (ICIT)*, Dec. 2022, pp. 982–987.
- [35] A. N. Ashik, M. S. H. Shanto, R. H. Khan, M. H. Kabir, and S. Ahmed, "Recognizing Bangladeshi traffic signs in the wild," in *Proc. 25th Int. Conf. Comput. Inf. Technol. (ICIT)*, Dec. 2022, pp. 1004–1009.
- [36] M. A. Rahman, N. I. Asad, M. M. H. Omi, M. B. Hasan, S. Ahmed, and M. H. Kabir, "FUSED-Net: Detecting traffic signs with limited data," 2024, *arXiv:2409.14852*.
- [37] J. L. Fleiss, "Measuring nominal scale agreement among many raters," *Psychol. Bull.*, vol. 76, no. 5, pp. 378–382, Nov. 1971.
- [38] F. Yang, G. Zamzmi, S. Angara, S. Rajaraman, A. Aquilina, Z. Xue, S. Jaeger, E. Papagiannakis, and S. K. Antani, "Assessing inter-annotator agreement for medical image segmentation," *IEEE Access*, vol. 11, pp. 21300–21312, 2023.
- [39] J. L. Leevy, T. M. Khoshgoftaar, R. A. Bauder, and N. Seliya, "A survey on addressing high-class imbalance in big data," *J. Big Data*, vol. 5, no. 1, pp. 1–30, Dec. 2018.
- [40] J. M. Johnson and T. M. Khoshgoftaar, "Survey on deep learning with class imbalance," *J. Big Data*, vol. 6, no. 1, pp. 1–54, Dec. 2019.
- [41] M. H. Rafi, M. R. Mahjabin, M. S. Rahman, M. H. Kabir, and S. Ahmed, "A critical analysis of deep learning applications in crop pest classification: Promising pathways and limitations," in *Proc. 26th Int. Conf. Comput. Inf. Technol. (ICIT)*, Dec. 2023, pp. 1–6.
- [42] S. R. Raiyan, Z. Z. Amio, and S. Ahmed, "HaSPeR: An image repository for hand shadow puppet recognition," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Jun. 2024, pp. 4446–4456.
- [43] T. Ahmed, M. B. Hasan, S. Ahmed, and M. H. Kabir, "ExE-Net: Explainable ensemble network for potato leaf disease classification," in *Proc. IEEE Can. Conf. Electr. Comput. Eng. (CCECE)*, Aug. 2024, pp. 335–339.
- [44] A. Herok and S. Ahmed, "Cotton leaf disease identification using transfer learning," in *Proc. Int. Conf. Inf. Commun. Technol. Sustain. Develop. (ICT4SD)*, Sep. 2023, pp. 158–162.
- [45] S. Ahmed, M. B. Hasan, T. Ahmed, and M. H. Kabir, "Dexnet: Combining observations of domain adapted critics for leaf disease classification with limited data," in *Proc. Asian Conf. Pattern Recognit.* Cham, Switzerland: Springer, 2025, pp. 130–146.
- [46] M. H. Ashmafee, T. Ahmed, S. Ahmed, M. B. Hasan, M. N. Jahan, and A. A. Rahman, "An efficient transfer learning-based approach for apple leaf disease classification," in *Proc. Int. Conf. Electr., Comput. Commun. Eng. (ECCE)*, Feb. 2023, pp. 1–6.
- [47] R. H. Chowdhury and S. Ahmed, "MangoLeafViT: Leveraging lightweight vision transformer with runtime augmentation for efficient mango leaf disease classification," in *Proc. 27th Int. Conf. Comput. Inf. Technol. (ICIT)*, Dec. 2024, pp. 699–704.
- [48] P. Kumar, R. Bhatnagar, K. Gaur, and A. Bhatnagar, "Classification of imbalanced data: Review of methods and applications," *IOP Conf. Ser., Mater. Sci. Eng.*, vol. 1099, no. 1, 2021, Art. no. 012077.
- [49] T. R. Fuad, S. Ahmed, and S. Ivan, "AQUA20: A benchmark dataset for underwater species classification under challenging conditions," 2025, *arXiv:2506.17455*.
- [50] S. Ivan, T. Ahmed, S. Ahmed, and M. H. Kabir, "A vision-language multimodal framework for detecting hate speech in memes," in *Proc. IEEE Can. Conf. Electr. Comput. Eng. (CCECE)*, Aug. 2024, pp. 464–468.
- [51] M. B. Hasan, T. Ahmed, S. Ahmed, and M. H. Kabir, "GaitGCN++: Improving GCN-based gait recognition with part-wise attention and DropGraph," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 35, no. 7, Jul. 2023, Art. no. 101641.
- [52] M. S. Morshed, S. Ahmed, T. Ahmed, M. U. Islam, and A. B. M. A. Rahman, "Fruit quality assessment with densely connected convolutional neural network," in *Proc. 12th Int. Conf. Electr. Comput. Eng. (ICECE)*, Dec. 2022, pp. 1–4.
- [53] M. A. Alanezi, M. S. Shahriar, M. B. Hasan, S. Ahmed, Y. A. Sha'aban, and H. R. Boucekara, "Livestock management with unmanned aerial vehicles: A review," *IEEE Access*, vol. 10, pp. 45001–45028, 2022.
- [54] A. Yasmeen, F. I. Rahman, S. Ahmed, and M. H. Kabir, "CSVC-Net: Code-switched voice command classification using deep CNN-LSTM network," in *Proc. Joint 10th Int. Conf. Inform., Electron. Vis. (ICIEV) 5th Int. Conf. Imag., Vis. Pattern Recognit. (icIVPR)*, Aug. 2021, pp. 1–8.

- [55] A. M. Khan, A. Ashrafee, R. Sayera, S. Ivan, and S. Ahmed, "Rethinking cooking state recognition with vision transformers," in *Proc. 25th Int. Conf. Comput. Inf. Technol. (ICCIT)*, Dec. 2022, pp. 170–175.
- [56] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *J. Big Data*, vol. 6, no. 1, pp. 1–48, Dec. 2019.
- [57] A. A. Rahman, M. B. Hasan, S. Ahmed, T. Ahmed, M. H. Ashmafee, M. R. Kabir, and M. H. Kabir, "Two decades of Bengali handwritten digit recognition: A survey," *IEEE Access*, vol. 10, pp. 92597–92632, 2022.
- [58] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [59] S. Ahmed, M. B. Hasan, T. Ahmed, M. R. K. Sony, and M. H. Kabir, "Less is more: Lighter and faster deep neural architecture for tomato leaf disease classification," *IEEE Access*, vol. 10, pp. 68868–68884, 2022.
- [60] T. Tajwar, M. Rahman, T. A. Chowdhury, S. Ahmed, M. Farazi, and M. H. Kabir, "Improving zero-shot semantic segmentation using dynamic kernels," in *Proc. Int. Conf. Digit. Image Computing: Techn. Appl. (DICTA)*, Nov. 2023, pp. 395–402.
- [61] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.
- [62] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4510–4520.
- [63] A. Howard, M. Sandler, B. Chen, W. Wang, L.-C. Chen, M. Tan, G. Chu, V. Vasudevan, Y. Zhu, R. Pang, H. Adam, and Q. Le, "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1314–1324.
- [64] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 8697–8710.
- [65] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [66] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [67] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2261–2269.
- [68] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2818–2826.
- [69] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, inception-resnet and the impact of residual connections on learning," in *Proc. AAAI Conf. Artif. Intell.*, 2017, vol. 31, no. 1, pp. 1–12.
- [70] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6105–6114.
- [71] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 11976–11986.
- [72] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1800–1807.
- [73] Z. C. Lipton, J. Berkowitz, and C. Elkan, "A critical review of recurrent neural networks for sequence learning," 2015, *arXiv:1506.00019*.
- [74] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [75] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," 2014, *arXiv:1406.1078*.
- [76] K. Yousaf and T. Nawaz, "A deep learning-based approach for inappropriate content detection and classification of YouTube videos," *IEEE Access*, vol. 10, pp. 16283–16298, 2022.
- [77] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
- [78] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proc. 14th Int. Conf. Artif. Intell. Statist.*, 2012, pp. 315–323.
- [79] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.

- [80] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th Int. Conf. Artif. Intell. Statist.*, vol. 9, 2010, pp. 249–256.



MUBTASIM KAMAL DIHAN received the B.Sc. degree in computer science and engineering from the Islamic University of Technology (IUT), Gazipur, Bangladesh, in 2025.

His research interests primarily include integrating blockchain with machine learning, particularly in developing intelligent and energy-efficient consensus mechanisms. Beyond this, his work also explores computer vision problems, such as video understanding, classification, and recognition, focusing on improving model generalization in complex visual scenes. In addition, he maintains an interest in game development and applying learning-based decision-making in interactive systems.



ABDULLAH received the B.Sc. degree in computer science and engineering from the Islamic University of Technology (IUT), Gazipur, Bangladesh, in 2025.

He has been involved in several research spanning fine-grained action recognition, human-centered reasoning, language modeling, and IoT-enabled blockchain technologies. His ongoing goal is to advance research in intelligent systems and contribute to the development of robust, transparent, and efficient AI-driven solutions. His research interests include machine learning, computer vision, human–computer interaction, natural language processing, and blockchain consensus.



AMINA received the B.Sc. degree in computer science and engineering from the Islamic University of Technology (IUT), Gazipur, Bangladesh, in 2025.

Her research interests include machine learning, transfer learning, action recognition, large language model reasoning, human–computer interaction, natural language processing, and blockchain consensus mechanisms. She has worked on several research projects in these areas, focusing on deep learning-based action recognition, reasoning frameworks for LLMs, and consensus algorithms. She aims to grow her knowledge and continue exploring these fields to make meaningful contributions to technology and research.



SABBIR AHMED received the B.Sc. and M.Sc. degrees in CSE from IUT, in 2017 and 2022, respectively.

He is an Assistant Professor with the Department of CSE, Islamic University of Technology (IUT). He is a member of the Computer Vision Research Group. Besides, he has worked with different applications of deep learning in the field of computer vision and NLP, such as leaf disease classification, traffic sign detection, meme understanding, text summarization, sentiment analysis, and applied research on LLMs. His current research is in improving few-shot learning algorithms for image classification tasks, and also in bias mitigation in large language models (LLMs). He was a Gold Medalist.

...